

Министерство науки и высшего образования Республики Казахстан

КАРАГАНДИНСКИЙ ИНДУСТРИАЛЬНЫЙ УНИВЕРСИТЕТ

Кафедра «Технологии искусственного интеллекта»

КУРСОВОЙ ПРОЕКТ

по дисциплине: Разработка приложений в C#

Тема: Игра «Брейкаут»

(оценка)

Руководитель: ст.преп.
Чванова А.О.

(подпись)

Члены комиссии:

Кан С.В

(подпись)

Авкурова Ж.С.

(подпись)

Выполнил: ст.гр ПИ-22
Ким Кирилл Викторович

Темиртау, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Анализ предметной области	3
1.1 Обзор существующих аналогов.....	4
1.2 Постановка задачи на разработку	7
2. Проектирование приложения.....	8
2.1 Спецификация требований.....	8
2.2 Описание алгоритмов	11
2.3 Описание структур данных.....	13
2.4 Работа с графикой, звуком, анимацией	14
3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	15
3.1 Руководство пользователя.....	15
3.2 Тестирование и отладка приложения.....	19
Заключение	20
Список использованных источников.	21
Приложение. Листинг программы	23

ВВЕДЕНИЕ

Современный мир насыщен различными формами развлечений и инновационными технологиями, включая игровую индустрию, которая занимает значительное место в жизни современного общества. Создание компьютерных игр является одним из наиболее динамично развивающихся и перспективных направлений в информационной технологии. Это обусловлено растущим спросом на качественные и увлекательные игровые продукты, которые не только развлекают, но и позволяют пользователю погрузиться в виртуальный мир с невероятными возможностями.

В рамках данного курсового проекта рассматривается разработка игры "Брейкаут", которая является классическим представителем жанра аркадных игр. Этот жанр имеет долгую историю и широкую популярность среди игроков разного возраста. "Брейкаут" сочетает в себе элементы логики, реакции, и управления, что делает её захватывающей и увлекательной для игрока.

Игровая индустрия продолжает активно развиваться, предлагая новые технологии и идеи для игровых проектов. Жанр аркадных игр, в который входит "Breakout", сохраняет свою популярность благодаря простоте и динамичности игрового процесса. С появлением новых технологий, таких как улучшенная графика, звуковые эффекты и возможности сетевой игры, игры в жанре "Breakout" приобретают новые возможности и привлекательность для игроков.

Цель курсового проекта:

Разработка игры "Breakout" с использованием языка программирования C# и платформы .NET Framework.

Задачи:

1. Создание игрового механизма для управления платформой и мячом.
2. Реализация логики столкновений и разрушения блоков.
3. Тестирование и отладка игры для обеспечения стабильной и удовлетворительной работы.

Данный курсовой проект имеет целью погрузиться в разработку игрового приложения с использованием современных технологий и методов программирования, а также расширить понимание и навыки в области разработки игровых проектов.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих аналогов

Брейкаут, выпущенный в 1976 году, является одной из самых популярных видеоигр всех времен. Его простой, но увлекательный игровой процесс, основанный на уничтожении платформ, вдохновил множество аналогов:

Arkanoid — это классическая игра в жанре Брейк-аут, созданная в 1986 году компанией Taito. Игрок управляет платформой, отскакивающей мячом, чтобы разрушить блоки на экране. Цель игры - очистить экран от блоков, не допустив утрату мяча. Игра сочетает в себе элементы логики, реакции и стратегии.



Рисунок 1.1- Arkanoid

Breakout Boost — это современная версия классической игры Брейкаут, выпущенная для мобильных устройств. Она сохраняет основной игровой механизм, но добавляет новые уровни, бонусы и графику, оптимизированную для сенсорных экранов.

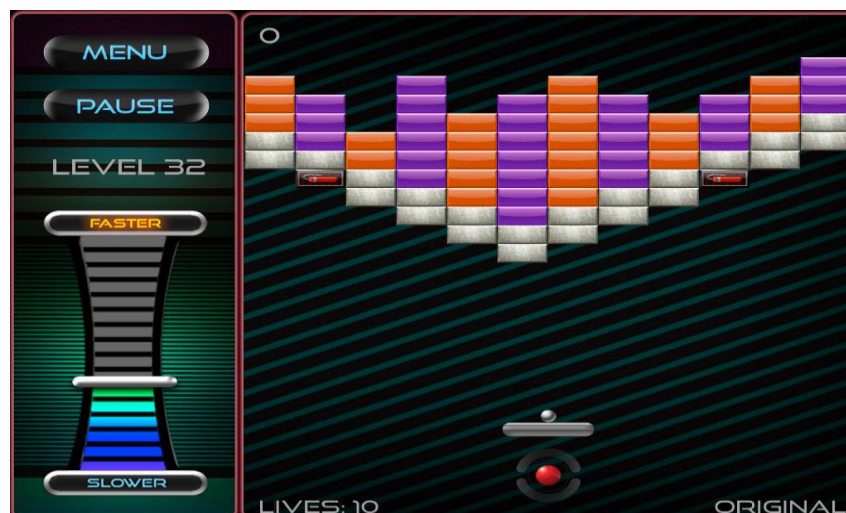


Рисунок 1.2 - Breakout Boost

Ricochet — это серия игр в жанре аркадного брейк-аута, разработанных и изданных компанией Reflexive Entertainment. Самая известная часть серии - Ricochet Lost Worlds, выпущенная в 2004 году.



Рисунок 1.3 – Ricochet

Brick Breaker Hero — это современная мобильная игра в жанре Брейк-аут с элементами ролевой игры. Игрок управляет героем, который использует различные способности и умения, чтобы разрушить блоки и спасти мир.

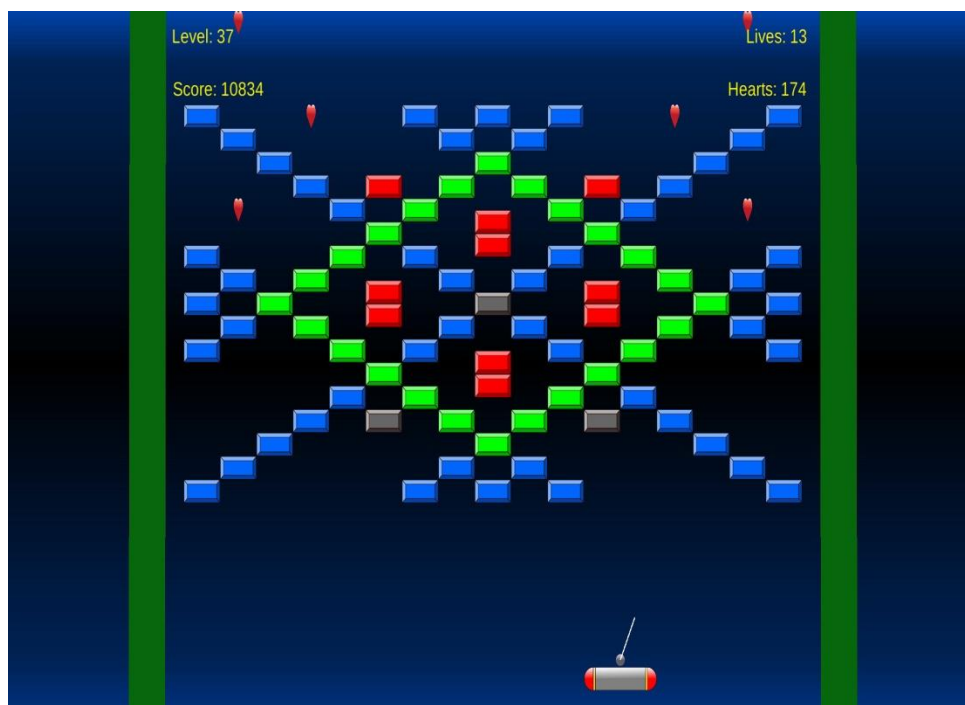


Рисунок 1.4 – Brick Breaker Hero

DX-Ball - это классическая игра в жанре "Breakout", которая предлагает уникальные уровни с различной конфигурацией блоков и бонусами. Игроку предлагается управлять платформой, чтобы отбивать мяч и разрушать блоки на экране.

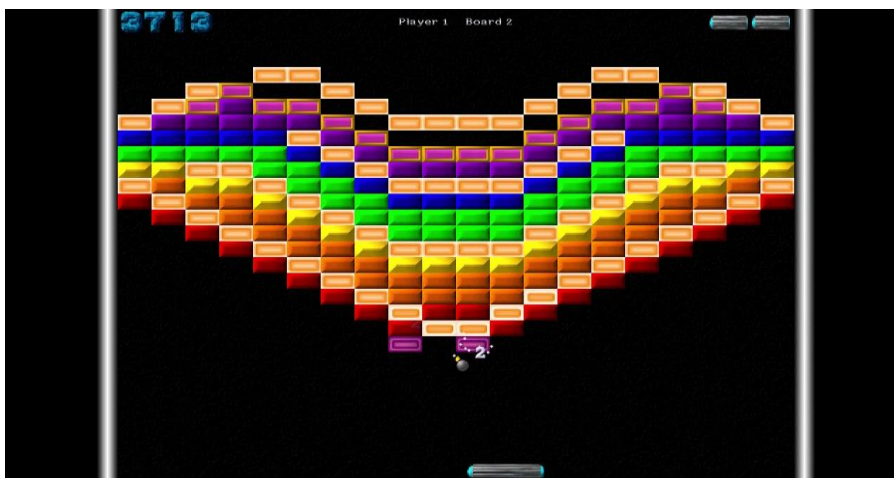


Рисунок 1.5 – DX-Ball

Magic Ball предлагает смесь классических механик "Breakout" с уникальными визуальными эффектами и специальными уровнями. Игроку предстоит управлять платформой, разрушать блоки и собирать различные бонусы на пути к успеху.



Рисунок 1.6 – Magic Ball

1.2 Постановка задачи на разработку

Для данного курсового проекта требуется изучить методы разработки компьютерных игр и создать версию игры "Брейкаут" на языке C# с использованием Windows Forms.

Можно выделить следующие этапы разработки:

1. Изучение методов и этапов разработки компьютерных игр.
2. Составление концепции и дизайна игры "Брейкаут".
3. Программирование основной логики игры с использованием языка C#.
4. Реализация графического интерфейса с помощью Windows Forms.
5. Тестирование и отладка игрового процесса.
6. Оптимизация и улучшение интерфейса пользователя.

Цель создания курсового проекта

Основной целью создания курсового проекта является разработка полнофункциональной версии игры "Брейкаут" с использованием технологий C# и Windows Forms. Этот проект позволит применить полученные знания в области разработки компьютерных игр на практике и приобрести опыт работы с графическим пользовательским интерфейсом.

Разработка игры "Брейкаут" включает следующие этапы:

1. Проектирование интерфейса игры и определение ее функциональных возможностей.
2. Создание основной логики игры, включая управление платформой и мячом, проверку на столкновение, касание и учет очков.
3. Разработка графического интерфейса с помощью элементов Windows Forms, включая игровое поле, платформы и отображение текущего состояния игры.
4. Реализация алгоритмов обработки пользовательского ввода для управления игровыми фигурами.
5. Тестирование игры на корректность работы и оптимизацию производительности.

Таким образом, целью данного проекта является создание полнофункциональной версии игры "Брейкаут" на языке C# с использованием Windows Forms, которая будет обладать удобным интерфейсом и приятным игровым опытом.

2. ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

2.1 Спецификация требований

1. Введение

1.1 Цель документа

Этот документ предназначен для определения требований к разработке игрового приложения "Брейкаут".

1.2 Контекст проекта

Игровое приложение "Брейкаут" разрабатывается в рамках выполнения курсового проекта по дисциплине "Разработка приложений в C#" по образовательной программе 6B06101 Программная инженерия.

2. Общие требования

2.1 Функциональные требования

2.1.1 Основные функции

Функция 1: Вывод на экран игрового поля.

Функция 2: Управление платформой.

Функция 3: Касание мяча.

Функция 4: Разрушение статичных платформ.

Функция 5: Конец игры: вылет мяча ниже управляемой платформы.

Функция 6: Конец игры: разрушение всех статичных платформ.

Функция 7: Проверка завершения игры.

2.1.2 Игровой процесс

Начало игры:

Игрок запускает приложение и начинает управлять платформой, отбивая мячом статичные платформы.

Ход игры:

Статичные платформы случайного цвета находятся над управляемой платформой белого цвета.

Игрок может управлять белой платформой, находящейся снизу перемещая её только влево и вправо.

При попадании мяча в платформы, он отскакивает от них, летя по траектории полета.

Окончание игры:

Игра заканчивается, когда будут разрушены все статичные платформы, или если мяч улетит ниже управляемой платформы.

2.2 Нефункциональные требования

2.2.1 Производительность

Частота кадров игры должна быть не менее 30 FPS.

Время отклика игры должно быть минимальным.

2.2.2 Совместимость

Поддержка платформ: Windows.

Разрешение экрана: минимум 800x600.

3. ТРЕБОВАНИЯ К ПОЛЬЗОВАТЕЛЬСКОМУ ИНТЕРФЕЙСУ

3.1 Основные экраны

Экран игры:

На этом экране отображается игровое поле.

Экран конца игры:

На этом экране отображается результат игры: поражение или победа и кнопка начала новой игры.

3.2 Элементы управления

Главный экран:

Кнопка "Enter": начинает новую игру.

Экран игры:

Управление с клавиатуры: стрелки влево и вправо для перемещения белой платформы

Экран конца игры:

Сообщение сверху: "Победа/Поражение! Нажмите Enter для запуска игры"

4. ДИЗАЙН И ПОЛЬЗОВАТЕЛЬСКИЙ ОПЫТ

4.1 Графический дизайн

Основные цвета:

Фон: черный.

Игровое поле: случайный контрастный цвет по отношению к фону, например, зеленый, синий и т.п.

Платформы: случайный цвет.

Дополнительные цвета:

Текст и управляемая платформа: белый по отношению к фону.

4.2 Пользовательский опыт

Интерфейс игры должен быть интуитивно понятным, позволяя игроку легко управлять платформой. Все элементы управления должны быть четко определены и доступны. Текст в игре должен быть читаемым. Игра должна реагировать на действия игрока мгновенно и плавно.

5. РАСПИСАНИЕ РАЗРАБОТКИ

5.1 Этапы разработки

№	Название этапа	Срок реализации	Результат
1	Выбор темы проекта	22.01.2024 – 26.01.2024	Название и описание игры для разработки
2	Обзор существующих аналогов	27.01.2024 – 17.02.2024	Анализ возможностей похожих игр, их функционала и интерфейса
3	Определение требований к проекту	06.02.2024 – 24.02.2024	Спецификация требований
4	Написание основных алгоритмов	15.02.2024 – 02.03.2024	Детальное описание алгоритмов в текстовом виде и в виде блок-схем
5	Выбор структур данных	26.02.2024 – 20.03.2024	Спецификация структур данных
6	Формирование общей архитектуры проекта	04.03.2024 – 05.04.2024	Набор отдельных составляющих проекта
7	Написание и тестирование программного кода	15.03.2024 – 12.04.2024	Работающее приложение
8	Оформление интерфейса приложения	19.03.2024 – 15.04.2024	Работающее приложение с дизайном

2.2 Описание алгоритмов

В данной части курсового проекта представлено описание основных алгоритмов, используемых в разработке компьютерной игры "Брейкаут". Каждый из представленных алгоритмов играет важную роль в функционировании игрового процесса, обеспечивая его правильное выполнение и создавая приятный игровой опыт для пользователей.

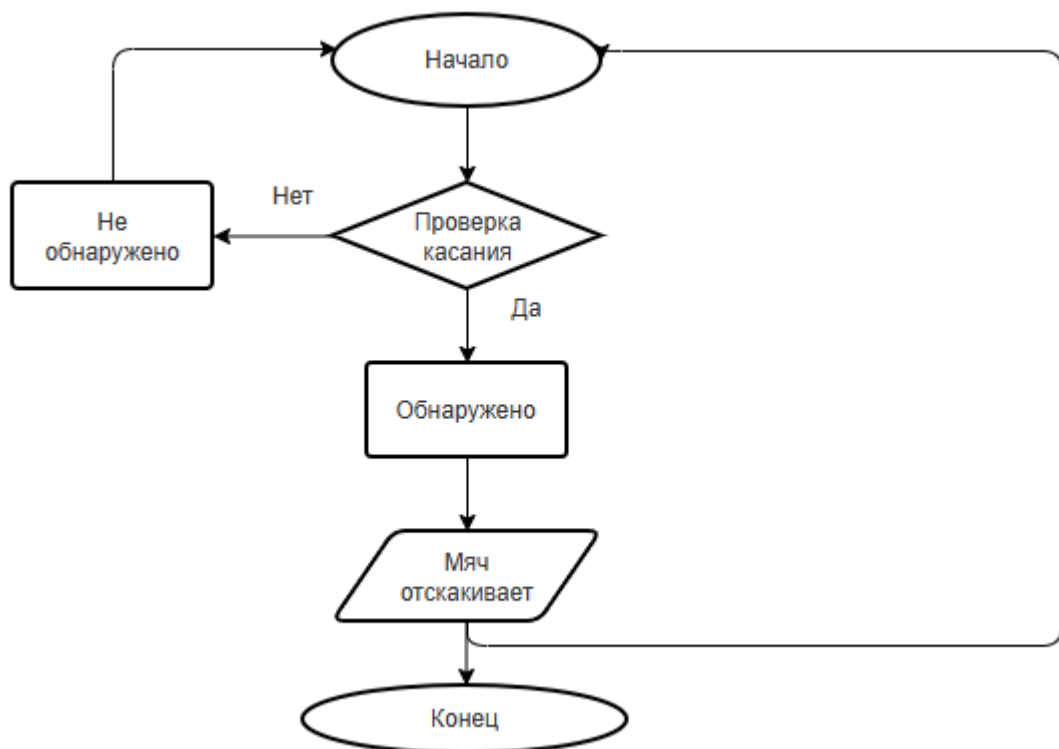


Рисунок 2.1 – Алгоритм обработки касания мяча

Алгоритм обработки касания начинается с первого касания к любой статичной платформе, где запускается процесс. Затем происходит проверка наличия столкновений между мячом и платформой на игровом поле. Если столкновение обнаружено, то мяч отскакивает, и летит от касающегося его платформы игрового поля, чтобы игровая сессия продолжалась. После этого алгоритм завершает свою работу, переходя к блоку "Конец". Если столкновение не обнаружено, то выполнение алгоритма просто завершается, и управление переходит к блоку "Конец", не происходя никаких дополнительных действий.

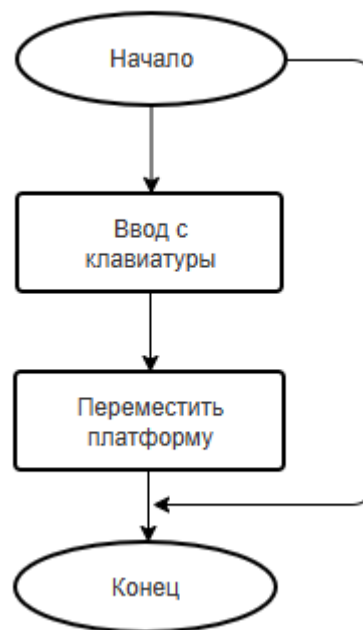


Рисунок 2.2 – Алгоритм управление платформой

Алгоритм управления платформы начинается с ввода с клавиатуры (стрелки влево/вправо), где начинается процесс перемещения платформы. После каждого действия до окончания игры алгоритм продолжает действовать.



Рисунок 2.3 – Алгоритм движения мяча

Алгоритм движения мяча в игре "Breakout" включает в себя инициализацию начальных параметров мяча, его непрерывное обновление координат в зависимости от текущего направления и скорости, проверку столкновений с игровыми объектами, обработку этих столкновений (включая изменение направления движения мяча) и проверку условий завершения игры, таких как падение мяча за нижний край игрового поля или достижение определенной цели.

2.3 Описание структур данных

Переменные и инициализация:

Для определения направления движения игрока и состояния игры используются флаги: goLeft, goRight, isGameOver.

Переменная для отслеживания счёта игрока: score.

За платформы, которые находятся на игровом поле, отвечает массив: blockArray.

Координаты мяча и его скорость по горизонтали и вертикали: ballx, bally

Скорость управляемой платформы: playerSpeed.

Методы:

Для инициализации игровых переменных, начальное положение игрока и мяча, запуск таймера игры: setupGame().

За обработку окончания игры отвечает: gameOver(string message).

Расстановка блоков на форме: PlaceBlocks().

Удаление блоков с формы: removeBlocks().

События и методы для управления игрой:

mainGameTimerEvent(object sender, EventArgs e): обработка таймера игры, движение игрока и мяча, столкновения, управление счетом и проверка условий завершения игры.

keyisdown(object sender, KeyEventArgs e): обработка нажатия клавиш для управления игроком.

keyisup(object sender, KeyEventArgs e): обработка отпускания клавиш и перезапуск игры при окончании игры.

2.4 Работа с графикой, звуком, анимацией

Игра "Брейкаут" использует различные компоненты и методы для реализации анимации и графического сопровождения, обеспечивая плавное перемещение мяча, обновление игрового поля и отображение информации о текущем состоянии игры.

Для графического сопровождения используются компоненты PictureBox, которые отображают игровые объекты, такие как мяч, игрок и блоки.

Для работы с графикой в игре используются следующие методы и свойства:

`pictureBox.Image`: Свойство, которое определяет изображение, отображаемое в PictureBox, в данном случае – это случайные цвета на платформах.

Для анимации использовалось изменение свойств объектов PictureBox, таких как `Left` и `Top`, для перемещения игровых объектов по экрану.

Таким образом, с использованием указанных компонентов и методов реализуется анимация и графическое сопровождение игры "Брейкаут".

3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

3.1 Руководство пользователя

"Breakout" - это классическая аркадная игра, в которой игроку предстоит управлять платформой и шаром. Цель игры – разрушить шаром платформы на игровом поле.

Основные цели и правила:

Цель игры – разрушить платформы мячом, которые находятся сверху управляемой платформы игрового поля. А также ловить мяч, отбивая своей платформой. Правила просты: игрок управляет платформой, отбивая мяч, чтобы заполнить разрушить платформы на игровом поле. Когда платформы все будут разрушены, игрок побеждает. Игра заканчивается, когда мяч будет потерян или будут разрушены все платформы на игровом поле.

Минимальные и рекомендуемые характеристики компьютера для запуска игры:

Минимальные требования:

- ОС: Windows 7/8/10
- Процессор: 1 GHz или эквивалентный
- Оперативная память: 128 MB RAM
- Видеокарта: совместимая с DirectX 9
- Место на жестком диске: 25 MB

Рекомендуемые характеристики:

- ОС: Windows 10
- Процессор: 2 GHz или более
- Оперативная память: 256 GB RAM или более
- Видеокарта: совместимая с DirectX 9
- Место на жестком диске: 25 MB

Порядок установки игры:

- Скачайте архив с игрой с надежного источника.
- Распакуйте архив в удобную для вас директорию на вашем компьютере.
- Запустите файл с игрой, который называется "Breakout.sln".

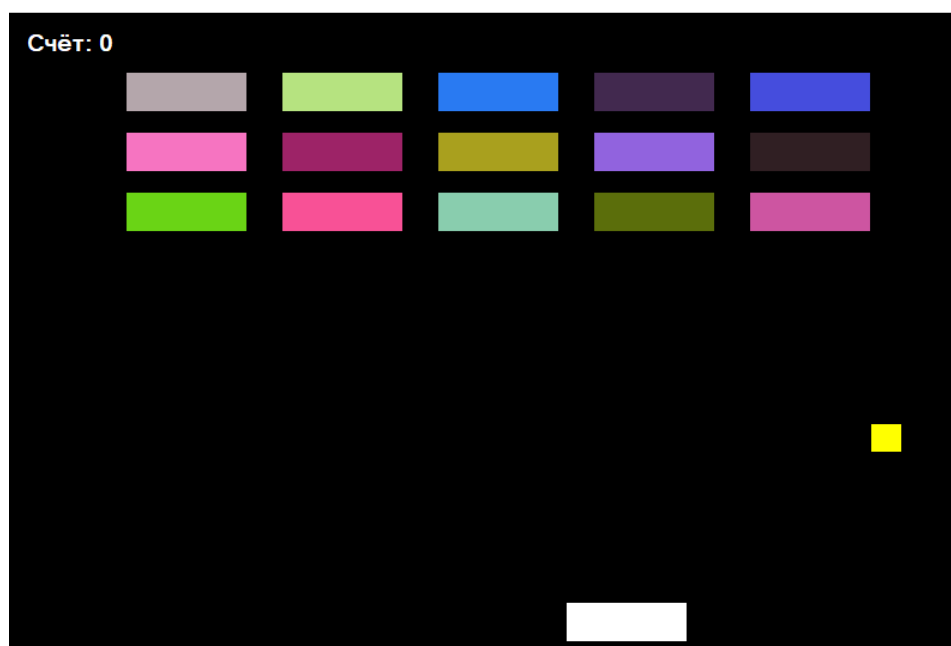


Рисунок 3.1 – Интерфейс

Интерфейс:

- Игровое поле: Сверху находятся статичные платформы, которые нужно разрушать шаром и представляют собой плиты разных цветов, где происходит основной игровой процесс - падение и перемещение фигур.
- Шар: Представляет собой желтый квадрат, летающий от стены или платформы по траектории.
- Управляемая платформа: Представляет собой белую платформу, которая отбивает шар, и управляется пользователем.
- Информация о счете: Расположена в левом верхнем углу, показывает информацию об разрушенных платформах и сообщение о начале игры.
- Управление: Происходит нажатием на клавиши клавиатуры. Кнопки стрелок позволяют перемещать управляемую платформу влево и вправо, что позволяет отбивать шар.



Рисунок 3.2 – Победа

Победа:

- Информация о счете: Показывает, что все платформы были разрушены и возможность начать заново игру.
- Шар: Шар остается в последней позиции, при разрушении последней платформы.
- Управляемая платформа: Аналогично шару остается в последней позиции при окончании игры.

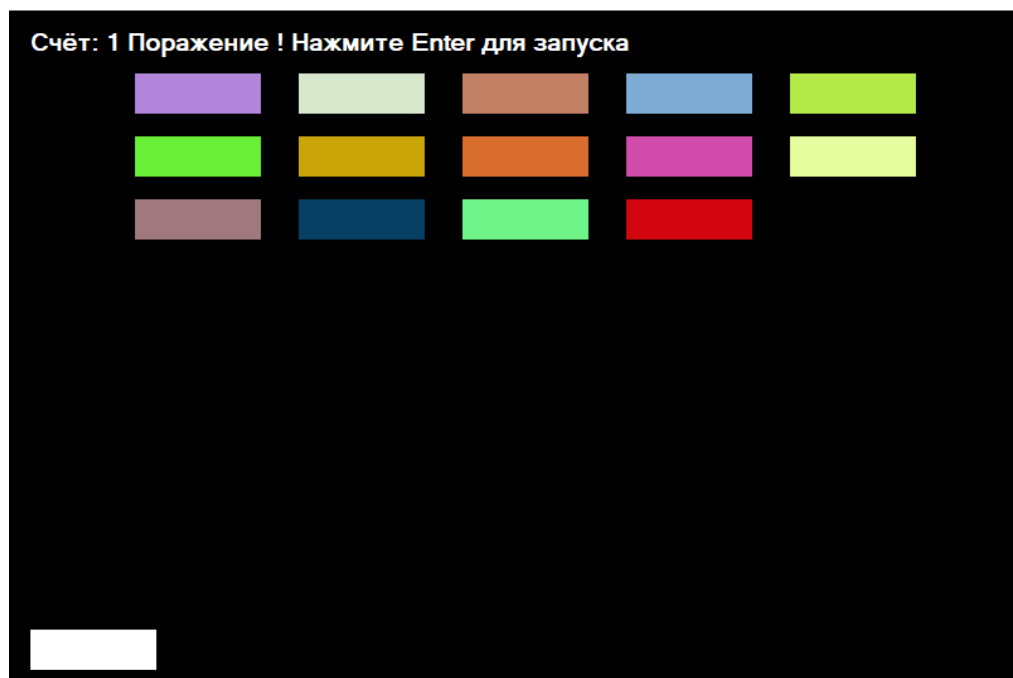


Рисунок 3.3 – Поражение

Поражение:

- Информация о счете: Показывает, что была уничтожена одна платформа, и шар улетел ниже управляемой платформы, возможность начать заново игру.
- Управляемая платформа: Остается в последней позиции при окончании игры.

Инструкции по запуску и использованию игры:

- Чтобы начать игру, необходимо запустить файл "Breakout.sln".
- Для выхода из игры, в любой момент, нажмите кнопку "Esc".
- Во время игры используйте клавиши управления (стрелки) для перемещения платформы.
- По завершении игры вы увидите свой результат и сможете начать игру заново.

Таблица 3.1 - Список сообщений

Сообщение	Причина, дальнейшие действия
Победа! Нажмите Enter для запуска	Это сообщение появляется при уничтожении всех платформ на игровом поле, необходимо нажать Enter чтобы начать заново игру.
Поражение! Нажмите Enter для запуска	Это сообщение появляется при потере шара на игровом поле, необходимо нажать Enter чтобы начать заново игру.
Счет: (от 0 до 15)	Меняется при каждом разрушении платформы. Одна разрушенная платформа равна 1

3.2 Тестирование и отладка приложения

№	Тестовые данные	Ожидаемый результат	Полученный результат	Тип ошибки	Исправление
1	Касание шара	Шар должен коснуться и отлететь от платформ	Шар разрушает платформы с дистанции не касаясь платформ	Синтаксическая ошибка	Добавлен код для корректной работы касания шара.
2	Проверка на столкновение со статичными платформами	Шар должен разрушить и отлететь по траектории	Шар пролетал сквозь платформы	Синтаксическая ошибка	Добавлен код для платформ, заполняющий picturebox.
3	Завершение игры	Отображение результата игры	После завершения счёт сбрасывался до нуля	Синтаксическая ошибка	isGameOver = true – исправил ошибку.

Заключение

В ходе выполнения данного курсового проекта была проведена разработка классической игры "Брейкаут" на платформе C# с использованием Windows Forms. Этот проект позволил не только закрепить теоретические знания по программированию на языке C# и работе с графическими интерфейсами, но и приобрести ценный практический опыт в области разработки игровых приложений.

В ходе проекта были достигнуты следующие результаты: успешная реализация основной игровой механики, включая управление платформой и шаром, а также правильное отображение игрового поля и элементов интерфейса. Была также осуществлена обработка основных игровых событий, таких как завершение игры при разрушении платформ на игровом поле. Помимо этого, разработка игры "Брейкаут" была очень увлекательной, уверен, что игры в данном направлении будут развиваться. Также, изучил в контексте управления движением и расположение игровых элементов. Также были приобретены навыки в организации кода, его структурировании и оптимизации для повышения производительности и улучшения читаемости.

Также, я научился пользоваться и создавать комфортный и удобный интерфейс, который поможет мне в дальнейшем создавать более серьезные и сложные решения в программировании.

В заключение, данный проект оказался не только интересным и познавательным, но и ценным этапом в процессе обучения программированию. Полученные знания и навыки будут полезны при дальнейшем изучении и разработке программного обеспечения.

Список использованных источников.

1. Microsoft Documentation. "Windows Forms в .NET" [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/desktop/winforms/?view=netdesktop-5.0> , свободный - (дата обращения: 07.04.2018).
2. Stack Overflow. "C# Ответы на вопросы" [Электронный ресурс]. - Режим доступа: <https://stackoverflow.com/questions/tagged/c%23>, свободный - (дата обращения: 12.09.2014).
3. Курс "Основы программирования на C#" на портале "Stepik" [Электронный ресурс]. - Режим доступа: <https://stepik.org/course/44229/syllabus> , свободный - (дата обращения: 13.04.2024).
4. "Статьи по созданию игр на C#" [Электронный ресурс]. - Режим доступа: <https://gamedev.ru/tutorials/?q=c%23> , свободный - (дата обращения: 26.05.2013).
5. "Форум по разработке игр на C#" на сайте GameDev.net [Электронный ресурс]. - Режим доступа: <https://www.gamedev.net/forums/forum/18-programming/> , свободный - (дата обращения: 21.01.2017).
6. "Примеры готовых игр на C#" на сайте itch.io [Электронный ресурс]. - Режим доступа: <https://itch.io/games/made-with-c-sharp> , свободный - (дата обращения: 11.07.2021).
7. "Справочник по языку программирования C#" на сайте Tutorials Point [Электронный ресурс]. - Режим доступа: <https://www.tutorialspoint.com/csharp/index.htm> , свободный - (дата обращения: 11.11.2011).
8. "Видеокурсы по разработке игр на C#" на платформе Udemy [Электронный ресурс]. - Режим доступа: <https://www.udemy.com/courses/search/?q=c%23%20game%20development> , платный - (дата обращения: 03.02.2020).
9. "Форум по разработке игр на C#" на сайте GameDev.net [Электронный ресурс]. - Режим доступа: <https://www.gamedev.net/forums/forum/18-programming/> , свободный - (дата обращения: 02.08.2014).
10. Чарльз Петцольд. "Программирование на C#. Полное руководство" [Электронный ресурс]. - Доступно на сайте издательства "Вильямс". - Режим доступа: <https://www.williamspublishing.com/> , платный - (дата обращения: 13.04.2010).
11. "Топ 50 игр, похожих на Arena Breakout на Андроид и IOS" – Режим доступа: <https://www.ranker.com/list/the-best-breakout-clone-games-of-all-time/kyle-townsend>, свободный - (дата обращения: 05.02.2015).
12. "Программирование на C# для начинающих. Основные сведения" Режим доступа: https://codelibs.ru/c_sharp/programmirovanie-na-c-dlya-nachinayushih-osnovnye-svedeniya/ - (дата обращения: 13.04.2024).

13. "Пишем игровую логику на C#. Часть 1/2" Режим доступа: <https://habr.com/ru/articles/322258/> - (дата обращения: 16.03.2011).
14. "Разработка игры на C# WPF. Создаем игру без игрового движка" - Режим доступа: <https://itproger.com/news/razrabotka-igri-na-c-wpf-sozdaem-igru-bez-igrovogo-dvizhka> , свободный - (дата обращения: 26.02.2016).
15. "How To Make A Simple Breakout Game In Windows Form With" Режим доступа: https://thewikihow.com/video_m18aCV-ox38?ysclid=lvdc0x5znb349047236 , свободный - (дата обращения: 26.02.2020).

Приложение. Листинг программы

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Купс
{
    public partial class Form1 : Form
    {

        bool goLeft; // направления движения игрока влево
        bool goRight; // направления движения игрока вправо
        bool isGameOver; // окончание игры

        int score; // счёт
        int ballx; // координаты мяча по горизонтали
        int bally; // координаты мяча по вертикали
        int playerSpeed; // скорость плеера

        Random rnd = new Random(); // Объект для генерации случайных чисел

        PictureBox[] blockArray;

        public Form1()
        {
            InitializeComponent();

            PlaceBlocks(); // Расставляем блоки на форме при загрузке игры
        }

        private void setupGame()
        {
            // Инициализация игровых переменных
            isGameOver = false;
            score = 0;
            ballx = 5;
            bally = 5;
            playerSpeed = 15;
            txtScore.Text = "Счёт: " + score;

            // Начальное положение мяча и игрока
            ball.Left = 376;
            ball.Top = 328;
```



```

player.Left = 347;

// Запуск таймера игры
gameTimer.Start();
// Случайная раскраска блоков на форме
foreach (Control x in this.Controls)
{
    if(x is PictureBox && (string)x.Tag == "blocks")
    {
        x.BackColor = Color.FromArgb(rnd.Next(256), rnd.Next(256), rnd.Next(256));
    }
}
}

private void gameOver(string message)
{
    // Обработка окончания игры
    isGameOver = true;
    gameTimer.Stop();

    txtScore.Text = "Счёт: " + score + " " + message;
}

private void PlaceBlocks()
{
    // Расстановка блоков на форме
    blockArray = new PictureBox[15];

    int a = 0;

    int top = 50;
    int left = 100;

    for(int i = 0; i < blockArray.Length; i++)
    {
        blockArray[i] = new PictureBox();
        blockArray[i].Height = 32;
        blockArray[i].Width = 100;
        blockArray[i].Tag = "blocks";
        blockArray[i].BackColor = Color.White;

        if(a == 5)
        {
            top = top + 50;
            left = 100;
            a = 0;
        }
    }
}

```

```

        if(a < 5)
        {
            a++;
            blockArray[i].Left = left;
            blockArray[i].Top = top;
            this.Controls.Add(blockArray[i]);
            left = left + 130;
        }

    }
    setupGame(); // Инициализация игры после расстановки блоков
}

private void removeBlocks()
{
    // Удаление блоков с формы
    foreach (PictureBox x in blockArray)
    {
        this.Controls.Remove(x);
    }
}

// Событие таймера игры
// ... (обработка движения игрока, мяча, столкновений и т.д.)
private void mainGameTimerEvent(object sender, EventArgs e)
{
    txtScore.Text = "Счёт: " + score;

    if (goLeft == true && player.Left > 0)
    {
        player.Left -= playerSpeed;
    }

    if (goRight == true && player.Left < 700)
    {
        player.Left += playerSpeed;
    }

    ball.Left += ballx;
    ball.Top += bally;

    if (ball.Left < 0 || ball.Left > 775)
    {
        ballx = -ballx;
    }
    if (ball.Top < 0)
    {
        bally = -bally;
    }

    if (ball.Bounds.Intersects(player.Bounds))

```

```

{
    ball.Top = 463;

    bally = rnd.Next(5, 12) * -1;

    if (ballx < 0)
    {
        ballx = rnd.Next(5, 12) * -1;
    }
    else
    {
        ballx = rnd.Next(5, 12);
    }
}

foreach (Control x in this.Controls)
{
    if (x is PictureBox && (string)x.Tag == "blocks")
    {
        if(ball.Bounds.Intersects(x.Bounds))
        {
            score += 1;

            bally = -bally;

            this.Controls.Remove(x);
        }
    }
}

if(score == 15)
{
    gameOver("Победа ! Нажмите Enter для запуска");
}

if(ball.Top > 580)
{
    gameOver("Поражение ! Нажмите Enter для запуска");
}

}
// Обработка нажатия клавиши вниз
// ... (установка флагов направления движения игрока)
private void keyisdown(object sender, KeyEventArgs e)
{
    if(e.KeyCode == Keys.Left)

```

```

    {
        goLeft = true;
    }
    if(e.KeyCode == Keys.Right)
    {
        goRight = true;
    }
}
// Обработка отпускания клавиши
// ... (сброс флагов направления движения игрока, перезапуск игры)
private void keyisup(object sender, EventArgs e)
{
    if (e.KeyCode == Keys.Left)
    {
        goLeft = false;
    }
    if (e.KeyCode == Keys.Right)
    {
        goRight = false;
    }
    if(e.KeyCode == Keys.Enter && isGameOver == true)
    {
        removeBlocks();
        PlaceBlocks();
    }
}

private void Form1_Load(object sender, EventArgs e)
{
}
}
}

```